

Minimum Window Substring

Given two strings s and t of lengths m and n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string $""$.

Input: $s = \text{"ADOBECODEBANC"}, t = \text{"ABC"}$

Output: "BANC"

Example 2:

Input: $s = \text{"a"}, t = \text{"a"}$

Output: "a"

Input: $s = \text{"a"}, t = \text{"aa"}$

Output: $""$

Brute Force

All possible substring,
check if t exists

TC: $O(m^2)$

Expand

$s = "ddaaabbbca"$ $t = "abc"$

$\uparrow$ s $\downarrow$ e

$d: 1-2$ $a: \cancel{x} \emptyset - 1-2$ $b: \cancel{x} \emptyset - 1$ $c: \cancel{x} \emptyset$

count = ~~0~~ ~~1~~ ~~2~~ 3

$$\text{len} = e - s + 1 = 8$$

Shrink

$s = "ddaaabbbca"$ $t = "abc"$

$\uparrow$ s $\downarrow$ e

$d: 1-2 \cancel{x} \emptyset$ $a: \cancel{x} \emptyset - 1-2-3 \emptyset 1$ $b: \cancel{x} \emptyset - 1$ $c: \cancel{x} \emptyset$
--

count = ~~0~~ ~~1~~ ~~2~~ ~~3~~ 2

$$\text{len} = e - s + 1 = 8$$

$$e - s + 1 = 7$$

$$e - s + 1 = 6$$

$$e - s + 1 = 5$$

$$e - s + 1 = 4$$

Expand

$s = "ddaaabbbca"$ $t = "abc"$

$d: \cancel{1} \cancel{2} 0$
 $a: \cancel{x} \cancel{0} \cancel{1} \cancel{2} \cancel{1} \cancel{0} 0$
 $b: \cancel{x} \cancel{0} 1$
 $c: \cancel{x} 0$

count = $\cancel{0} \cancel{1} \cancel{2} 3 \cancel{2} 3$

len = $e - s + 1 = 4$
 $e - s + 1 = 4$

Shrink

$s = "ddaaabbbca"$ $t = "abc"$

$d: \cancel{1} \cancel{2} 0$
 $a: \cancel{x} \cancel{0} \cancel{1} \cancel{2} \cancel{1} \cancel{0} 0$
 $b: \cancel{x} \cancel{0} \cancel{1} \cancel{0} 1$
 $c: \cancel{x} 0$

count = $\cancel{0} \cancel{1} \cancel{2} \cancel{3} \cancel{2} \cancel{3} 2$

len = $e - s + 1 = 4$
 $e - s + 1 = 3$

Shrink

$s = "ddaaabbbca"$ $t = "abc"$

len = $e - s + 1 = 3$

```
String window (s, t) {  
    if (s.len < t.len)  
        return ""
```

```
    int[] need = new int[128];  
    for (char ch: t.toCharArray()) {  
        need[ch]++  
    }
```

```
    int start = 0;  
    count = t.length;  
    minLen = ∞;  
    int l = 0, r = 0;
```

```
    while (r < s.length()) {  
        rch = s.charAt(r)  
        if (need[rch] > 0) {  
            count --  
        }  
        need[rch] --  
        r++  
        while (count == 0) {  
            if (r - l < minLen) {  
                start = l  
                minLen = r - l  
            }  
            lch = s.charAt(l)  
            need[lch] ++  
            if (need[lch] > 0) {
```

```

    {
        count ++
    }
    l++;
}

```

```

} return minLen == ∞ ? "" : s.substring(start,
                                           start + minLen)

```

TC: $O(m+n)$

SC: need [128]
 $O(1)$